TITLE

UNDERGRADUATE FINAL YEAR PROJECT

Hans Sebastian
2022390007

Computer Science
Faculty of Engineering annd Technology
Sampoerna University
Jakarta
2025

TITLE

UNDERGRADUATE FINAL YEAR PROJECT

Hans Sebastian
2022390007

Supervisor 1: **Supervisor 2:**

Supervisor 1 Name with title **Supervisor 2 Name with title**

2025

CHAPTER II

INTRODUCTION

2.1 Background

Since the beginning of the information age, the use of computer aided design (CAD) tools and software have largely replaced the use of mylars and blueprints which require manual drawings and measurements, which can be prone to human errors and are costly and hard to reproduce. This is especially true in engineering-related fields with one study showing up to 50% reduction in task duration (Rahman et al., 2019**general_purpose_cad_simon**). Other benefits include the ability for immediate changes even during late-stage development which helps limit delays (Aranburu et al., 2021). The main benefit of CAD tools is ability to reutilize/repurpose existing CAD designs to be reused in other CAD projects, resulting in more efficient, reliable workflows that eliminates the need to redesign already existing components from scratch (**general_purpose_cad_simon**Mandelli & Berretti, 2022).

Although the use of CAD have improved the overall design and manufacturing process, they also pose their own set of issues. For example, CAD software developers often employ their own proprietary file formats, each with their own complex structures. This results in major incompatibility issues as users might have different tooling preferences. Other issues include difficulties in documentation and indexing for existing CAD designs, as the complexity of the file's structure inhibit the use of automated annotation and indexing tools, forcing the use of manual annotation. As a result, numerous universal CAD file formats such as STEP and STL have been developed to handle the issue of compatibility. However, issues regarding documentation and indexing still persists, prompting the research for potential solutions.

2.2 Problem Formulation

There are multiple issues relating to CAD design annotation, one example being that manual annotation incurs an unnecessary overhead, diverting manpower and time away from the main design process. Furthermore, manual annotation also introduce risks regarding annotation quality and accuracy, as human annotators might miss important details or mislabel important components, reducing overall workflow efficiency (Mandelli & Berretti, 2022). Difficulties in determining annotation formats and standards increases risk of errors, which is especially noticeable when dealing with third-party designs.

The development of artificial intelligence (AI) have resulted in new techniques for automating the process, however most AI-based methods and models require the use of additional conversion steps into images and/or point clouds to leverage existing image processing and computer vision techniques (Mandelli & Berretti, 2022). This added steps reduce the overall efficiency and increases the risk of misidentification/misclassification. This study aims to develop a novel method for processing CAD files which preserves the existing semantic context and connections between the parts without the need for additional conversion steps into visual representations. Additionally, current state-of-the-art (SotA) models such as Gemini 3 and Chat-GPT 5.1 overly rely on the existing metadata contained in the file, completely bypassing the geometry of the design. Other issues related to these models include privacy and latency concerns due to the necessity for an internet connection, as well as high computational and energy costs as the models are designed for general use (**<empty citation>**).

2.3 Research Objectives

Based on the topic discussed in the Problem Statement section, this study aims to complete the following research objectives:

1. Generate accurate description for given CAD STEP file using novel transformer-based model.
2. Bridging the modality gap between human language and 3D geometric data (points, lines, etc).
3. Enhance CAD file indexing by leveraging model-generated descriptions as semantic metadata.
4. Enable low latency inference using compact, topology-aware tokenizer.

2.4 Significance of Study

This study's significance and contributions towards existing literature are detailed as follows:

1. Enable faster development and project organization with automatic STEP CAD file annotation.
2. Develop a topology-first serialization strategy for STEP file B-rep graph
3. Advance existing classification models from general classification to generating detailed descriptions.

2.5 Limitation of Study

The following study lists the following limitations and potential improvements:

1. **Context-dependence:** The labeling of CAD objects relies on designer intent, which might be lost during pre-processing.

2. **Implicit Bias:** The correctness and quality of the output and training process are determined based on human responses, which introduces potential bias.
3. **Limited Vocabulary:** The model's output and training data are based solely on the English language, limiting its ability to generate annotations in different languages.
   Additionally, the model's vocabulary is modified to reduce subjectivity by limiting the use of adjectives such as "big" or "small".
4. **Sparse classes:** The model's training data only consists of general common objects which can be easily found in public datasets in order to maximize the accuracy of the model. This is due to the scarcity of complex CAD designs in such datasets as they contain mostly reusable assets.

2.6 Organization of the report

The rest of the paper will be organized according to the following structure: Chapter II analyses the existing literature and basic terminologies required to understand the topic. Chapter III details the methodology used to gather the training dataset and the design considerations related to model development, from tokenizing, fine-tuning, to the benchmarking process. Chapter IV lists the results gathered during the benchmarking process paired with detailed analysis of said results. Lastly, Chapter V summarizes the conclusion obtained from the study as well as potential improvements for future research.

CHAPTER III

LITERATURE REVIEW

3.1 Data and Information

3.1.1 Definitions

Data refers to a collection of raw, unlabelled information which represents a set of truth/facts about the world (Olson, 2021). Data comes in two main categories, quantitative and qualitative (Hackman et al., 2024). Quantitative data refers to data which has a distinct form, with unique categories and can be analyzed using objective numerical operations and methods, where qualitative data lacks a concrete structure and is highly context-dependent and subjective (Schoonenboom, 2023). Quantitative data mostly consists of values that can be represented numerically (a person's height/weight, price of an object), while qualitative data refers to data about a subject such as nominal descriptions (a person's name, a novel's synopsis). This paper focuses on digital data, which is data represented as a sequence of symbols, mainly binary digits.

The concept of data differs from information in that data serves as the foundation from which information is generated (Hackman et al., 2024). In essence, information is a specific interpretation of a given dataset, derived from their type and values, as well as their context and semantic meaning. In particular, the context in which a collection of data is acquired determines the set of valid interpretations of said data, which in turn affects the possible inferences i.e. potential information gathered from it (Mason, 2019). Raw data in itself cannot be used directly, as it merely represents the attributes of a certain subject, without assigning to it any valid implication/insight. To make use of data, it needs to be converted into information through analysis and contextualization (Borrohou et al., 2025).

As the name suggests, information is the basis of all modern information technologies and systems such as search engines, recommendation systems, and artificial intelligence. These systems utilize information gathered from large datasets to gain knowledge, derive solutions and make decisions such as giving a custom-tailored recommendation to particular set of users in the case of recommendation systems, and generating responses in the case of AI. The amount of data generated daily has grown exponentially in the information age, with one estimate stating approximately 402.74 million terabytes are generated by electronic devices around the world daily, and an estimated 394 zettabytes will be generated by 2028 (Taylor, 2025). This prompts the need to develop a method

for organizing, analyzing, and managing large batches of data efficiently and accurately.

3.1.2 Data classification

Data classification is the process of assigning labels to data for better management purposes (Newhouse et al., 2023; Shivayogi, 2025). In short, data classification is the process of grouping data into different categories according to their characteristics for later processing and bookkeeping. Data classification allows for enhanced protection, security and risk management of data based on sensitivity and regulatory standards (Lane & Adelusi, 2023; Nguyen & Cuong, 2025; Newhouse et al., 2023). Furthermore, data classification allows for better organization and retrieval of certain types of data, improving the accuracy and performance of information gathering. Data classification methods largely follows this specific sequence of steps: 1) defining classification criteria, 2) data discovery and sensitive data detection and 3) data labeling based on type, usage, and security requirements (Shivayogi, 2025).

Most forms of data, such as text, images, and video/audio recordings, are categorized as qualitative as they lack a formal, fixed set of attributes and structure. Furthermore, data with complex structures such as CAD files contain a large amount of features and attributes that form deep semantic relationships. As a result, these types of data cannot be easily classified using conventional methods as they rely on assumptions regarding the data's structure. Recent advancements in artificial intelligence systems such as natural language processing and deep learning have allowed for reliable, efficient classification of these datatypes (Blessing, 2021). In modern systems, vector embeddings have become the cornerstone that enables these capabilities, replacing the less efficient rule-based approach in natural language processing, and acting as the basis for deep learning systems such as convolutional/recurrent neural network models (Patil et al., 2023; Asudani et al., 2023). This paper will focus on the use of vector embeddings as a method for enumerating and processing CAD files for classification purposes.

3.2 Vector Embeddings

3.2.1 Definition and Purpose

Vector embedding is defined as a method for converting and representing complex, unstructured data as a sequence of numerical values (Pajo, 2025; Chitturi, 2024). A vector embedding aims to create a compact, structured representation of an underlying datapoint while preserving the underlying context and semantic meanings contained in it (Chitturi, 2024). In other words, vector embeddings encode unstructured, context-rich data into points in an n-dimensional space, capturing the relationships of different data points in numeric form, allowing the data to be processed and analyzed using mathemathical operations (Pajo, 2025). This representation also renders the concept of "distance" as a quantifiable component in data analysis, where distance refers to semantic proxim-

ity/closeness in meaning (Kukreja et al., 2023). In short, the distance between n-dimensional vector representation of two datapoints closely models the semantic relationship between the original data.

Vector embedding serves as a way to efficiently encode large datapoints such as texts, images, video and audio into a digestible, compact format that could be easily processed by other systems. Vector embeddings also allows data to be searched, indexed, and grouped using their semantic similarity (Taipalus, 2024). One use case is in large language models, where models could be equipped with the ability to access external documents and data, allowing it to produce reliable output grounded on falsifiable truth (Omolayo & Babatope, 2025).

Other similar methods for encoding qualitative, complex data have been developed, such as one-hot encoding, which encodes each datapoint as states represented as a sequence of n-bits, where all but one bit is set to '0', and the '1' bit represents the current state of the system (Samuels, 2024). Another example is Bag of Words (BoW), which represents text as a "bag"; an unordered, unstructured collection of its words, or keypoints in the case of an image (Qader et al., 2019), and Term Frequency-Inverse Document Frequency (TF-IDF), which encodes words/keypoints as its number of occurrence and rarity inside a document (Qaiser & Ali, 2018).

Nonetheless, the methods listed previously face several limitations when compared to newer techniques such as vector embedding, such as the loss of semantic information such as the word order, grammatical structure, and other relevant context stored in the original. Other problems include the problem of dimensionality, which states that as the number of dimensions increase for a given set of data, the amount of available data becomes increasingly sparse and patterns more obscure (Banks & Fienberg, 2003). This limitation also massively increases the compute power and memory required to store and process these encodings while also increasing the risk of errors (Trunk, 1979). Vector embedding sidesteps these limitations by mapping similar items to vectors which have a close distance to each other, preserving semantic data and minimizing the number of features required. The shift from sparse, count-based encoding methods towards a more compact, learned encoding technique enables critical performance improvements in terms of data processing metrics such as compute time and output accuracy.

3.2.2 Embedding Models

Although vector embedding offers several important advantages, such advantages require specialized, novel machine learning models that are capable of distilling complex relationships between datapoints into a compact dense vector representation while minimizing the amount of features/dimensions. These models capture context ingrained within datapoints by learning how to assign a connection between two neighboring words/parts of data (Tran, 2024, Chap-

ter 4.1).

Most embedding models could be categorized into two groups: static models that generate a single, fixed representation for a word/data segment, such as continuous BoW and Subword models (Wang et al., 2019), and contextual models such as Bidirectional Encoder Representations from Transformers (BERT) and its variants that generate unique representations of the same word depending on the current context (Liu et al., 2025; Devlin et al., 2019). In recent times, contextual models have largely dominated due to their flexibility and better capabilities in terms of capturing the nuance and implicit interconnections between datapoints and its components. One popular example of this is the transformer architecture, which forms the basis the paper's research.

3.3 Transformer Architecture

3.3.1 Architecture Overview

Current state-of-the-art (SotA) embedding models are based on the transformer architecture, which acts as the foundation for commercial generative AI models such as ChatGPT, Gemini, DeepSeek, etc. Introduced in the seminal paper "Attention Is All You Need" by Vaswani (Vaswani et al., 2017), instead of processing data sequentially, the architecture utilized a novel approach for sequence-processing tasks referred in the paper as self-attention, replacing the recurrent, in-order structures of previous models like recurrent neural networks (RNNs) and long short-term memory models (LSTMs) (Vaswani et al., 2017; Sherstinsky, 2020; Hochreiter & Schmidhuber, 1997).

The models share their similarity in their use of tokenization, a process of dividing input data into smaller chunks referred to as tokens. The tokens are represented as numeric vectors, reducing the amount of storage and compute needed for further processing. The literature dictates two main paradigms of tokenizing input data, full word and subword tokenization. Full word tokenization, as it implies, splits input data based on a pre-defined separator, such as whitespace in text. The above method have largely been replaced by its counterpart, which divides input data into smaller subparts that has no fixed separator. Examples of subword tokenization include byte pair encoding and WordPiece (Suyunu et al., 2024; Kozma & Voderholzer, 2024; Devlin et al., 2019). The tokens generated from the process are then used as the main input data of the self-attention mechanism.

The self-attention mechanism detailed in the transformer architecture improves upon previous systems by enabling models to evaluate the importance of the surrounding parts of an input sequence, regardless of its ordering. For instance, consider the sentence "The dog barked at the cat to drive it away." Using self-attention, transformer-based models can correctly associate the word "it" with the word "cat" rather than "dog" (Vaswani et al., 2017). This capability of handling remote dependencies and understand context allows it to outperform previous architectures. The details of the mechanism lies in the

novel components that make up the architecture, forming a deep network. In particular, the transformer model's functionality and reliability is the outcome from combining two key components, encoders and decoders.

3.3.2 Encoder-Decoder

The encoder-decoder components consists of several blocks/subcomponents that in turn form the basis of a transformer layer, which are then stacked continuously to build the complex models. The encoder functions as a processing layer for the entire input sequence, aiming to build a rich, contextualized numerical representation utilizing the self-attention mechanism. The encoder is implemented as two main parts:

1. **Multi-Head Self-Attention:**
   The self-attention of the transformer is employed as different units/heads working in parallel, allowing the model to learn different types of relationships from the input data simultaneously (Voita et al., 2019). Each "head" is tasked with learning a different aspect of the given sequence. For instance, one head might focus on syntactic relationships while another learns semantic ones. This functionality is done through projecting the input data into different representation subspaces and applying self-attention in parallel. The final result is a vector generated by applying a linear transform to the concatenation of the heads' output (Vaswani et al., 2017)

2. **Position-wise Feed-Forward Networks (FFN):**
   The next step consists of passing the token vectors gathered and weighted from the attention mechanism through a feed-forward neural network. The subcomponent consists of two linear neural network layer with a non-linear activation function in between, such as ReLU. It enriches each token's representation by first expanding the vector's dimension (i.e. 512 to 2048), applying an activation layer and resizing the vectors back into its original size. This process allows the model to extract deep, interconnected patterns hidden within the data. Furthermore, the network acts as a key-value store for patterns learned during training, and its output represents a distribution of tokens containing said patterns (Geva et al., 2021).

The parallel nature of self-attention makes it difficult to retain information regarding sequence ordering. Hence, positional encodings in the form of special vectors are added to the embedddings to mitigate this limitation. The positional encoding vectors are generated using sinusoidal functions with different frequencies that represent positions of tokens inside the sequence (Vaswani et al., 2017). After the encoding process, the result generated by the encoder is passed through a decoder that remaps the resulting vectors back into tokens that forms the final output sequence. The decoder achieves this through three steps:

1. **Masked Multi-Head Self-Attention:**
   The first step of the decoding sequence is similar with encoding, with a key difference being he lack of a lookahead for future tokens. This is

done to prevent model overfitting as knowing future tokens might embed irrelevant connections into the model.

2. **Cross-attention:**
   Another change made in the decoding process is the addition of a cross-attention layer. The layer acts as a critical link between the encoder and decoder that functions as a window for the decoder to look at the encoder's final output. The encoder uses its internal state as a query in the form of a vector and relates it to the vector set generated by the encoder. This process enabels the decoder to determine the most relevant parts of the input and its relation to the final output (Vaswani et al., 2017).

3. **Position-wise Feed-Forward Networks (FFN):**
   The final decoding step is similar to the encoding process with no changes.

Enabling effective training of densely-layered networks require proper integration of the encoder and decoder. To meet these requirements, two operations are applied post-encoding and decoding to ensure stability and effective training. The integration involves adding a residual connection layer that adds the original input vector with the output to minimize the vanishing gradient problem. Next, the vectors are normalized and stabilized, ensuring a smoother and reliable training process (Vaswani et al., 2017). The architectural decisions that make up the model allows the connections between each token to be filtered and strengthened based on the input's context, and allows the model to retain more information from its training data (Vaswani et al., 2017).

3.4 Related Works

Early attempts at classifying CAD files mainly used feature descriptors such as Zernike and Reeb graph descriptors to determine the category of a CAD model based on shape and function (Ip & Regli, 2005).

Several other works focuses on 2D based methods such as utilizing machine learning to classify over 1600 CAD files using random forest, support vector and fully-connected neural network models by identifying 45 different basic shapes (Machalica & Matyjewski, 2019), with subsequent approaches combining GNN to measure and group CAD files based on their similarities in geometry (Roj et al., 2021). More examples of this type include RotationNet, which utilizes images of the same 3D objects from different angles to estimate its pose and category (Kanezaki et al., 2018).

Different category of related studies uses a 3D-based approach, with one leveraging the use of 3D grid representation (voxels) of the CAD file and applying a 3-dimensional-CNN that could detect and categorize features (Maturana & Scherer, 2015), and another applies a Mask-recurrent CNN model to extract 3D objects from a single 2D RGB image which can then be classified (Gümeli et al., 2022). Works such as PointNet utilizes 3D point clouds to estimate the class of an object (Qi et al., 2016). A different classification uses a custom 3D representation named PANORAMA and feeding the result to a CNN to find

keypoints and classify the CAD based on the detected features (Papadakis et al., 2009; Sfikas et al., 2017; Sfikas et al., 2018). Newer 3D-based approaches utilizes graph-based techniques to make use of the graph-like structure of CAD files such as STEP, with one work using Graph-based Neural Network (GNN) to categorize certain CAD STEP files into 7 different classes (Mandelli & Berretti, 2022). However, a research gap in previous works can be identified, since the cited works are limited to classifying a fixed class of CAD objects, such as bolts, and cannot be used to create a general description of objects not specified in the class dataset.

Another method applies a Feature Directed Acyclic Graph (FDAG) to determine the dependency between the shapes of CAD models which are then optimized until only the fundamental features and shapes remain. The shapes can then be compared to CAD models to determine their resemblance to the shapes used as a query (Li et al., 2011). One graph based method also utilizes the structure of STEP files to generate an attribute graph which can then be used as a measure for shape similarity (El-Mehalawi & Allen Miller, 2003a; El-Mehalawi & Allen Miller, 2003b). Although the methods above are able to determine other objects with similar shapes, the method relies on the user's knowledge of the general shapes that define each object category. This paper aims to fill identified research gaps mentioned through implementing a novel framework for analyzing CAD files using transformer based architecture.

CHAPTER IV

PROPOSED METHODOLOGY

4.1 Data Gathering

4.1.1 Dataset Source Selection

Publicly available datasets for CAD file designs can be found on the internet. One prominent example is the ABC dataset containing ..., Fusion 360 gallery containing ..., and Text2CAD which contains around 170.000 CAD models with 660.000 annotations ranging from short descriptions to detailed explanations (**text2cad_mohammad**). Other sources include free 3D/CAD design websites such as GrabCAD and TraceParts, which contain various CAD objects not found in compiled datasets. A summary of the comparison for the sources listed above is contained in Table 4.1 below.

Based on the analysis detailed in Table 4.1, this paper will utilize a combination of CAD designs contained in Text2CAD as well as other online sources such as ABC dataset and TraceParts to help improve dataset diversity and reduce the potential risks for low-quality designs that could negatively affect the model's output. To help minimize discrepancies between annotations of several different datasets, a standard for annotation format mimicking the existing annotations in the Text2CAD dataset will be used.

4.1.2 Dataset Preprocessing

4.1.2.1 Filtering

The dataset gathered from the selected sources are filtered to remove rare, uncommon and/or low quality CAD designs that could negatively affect the model's output. The dataset is first analyzed using Python with Pandas module and manual inspection to figure out types of CAD designs with low occurrences (less than 100 unique designs). The number is chosen according to the existing dataset size as well as model parameters which will be discussed in Section 4.2.

4.1.2.2 Annotation Format

Source	Type	Summary	Advantages	Disadvantages
TraceParts	Online Website	A website containing free CAD designs for standard parts required for engineering purposes complete with names.	Completely free, contains standardized names with detailed specifications and assured quality	Contains only engineering designs, requires complex web scraping techniques
GrabCAD	Online Website	A website containing 6+ million free CAD designs from online users for various applications	Completely free, better diversity in terms of CAD designs	CAD designs are sourced from anonymous users with unverified quality, require complex web scraping techniques
Text2CAD	Compiled Dataset	A collection of CAD files of different engineering parts with standardized annotation		
Fusion 360	Compiled Dataset			
ABC	Compiled Dataset			

Table 4.1: Comparison between various dataset sources.

4.2 Model Design

4.2.1 Tokenizer

4.2.2 Model Parameters

4.3 Training and Backtesting

BIBLIOGRAPHY

Aranburu, A., Justel, D., & Angulo, I. (2021). Reusability and flexibility in parametric surface-based models: A review of modelling strategies. *Computer-Aided Design and Applications*, *18*, 864–874. https://doi.org/10.14733/cadaps.2021.864-874

Asudani, D. S., Nagwani, N. K., & Singh, P. (2023). Impact of word embedding models on text analytics in deep learning environment: A review. *Artificial Intelligence Review*, *56*, 1–81. https://doi.org/10.1007/s10462-023-10419-1

Banks, D. L., & Fienberg, S. E. (2003). *Data mining, statistics* (R. A. Meyers, Ed.; Third Edition). Academic Press. https://doi.org/https://doi.org/10.1016/B0-12-227410-5/00164-2

Blessing, M. (2021). Ai-driven data classification and organization.

Borrohou, S., Fissoune, R., & Badir, H. (2025). The role of data transformation in modern analytics: A comprehensive survey. *Journal of Computer Languages*, *84*, 101329. https://doi.org/https://doi.org/10.1016/j.cola.2025.101329

Chitturi, K. (2024). Understanding vector embeddings in ai search: A technical deep dive. *INTERNATIONAL JOURNAL OF COMPUTER ENGINEERING & TECHNOLOGY*, *15*, 1725–1733. https://doi.org/10.34218/IJCET_15_06_147

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019, June). BERT: Pre-training of deep bidirectional transformers for language understanding. In J. Burstein, C. Doran, & T. Solorio (Eds.), *Proceedings of the 2019 conference of the north American chapter of the association for computational linguistics: Human language technologies, volume 1 (long and short papers)* (pp. 4171–4186). Association for Computational Linguistics. https://doi.org/10.18653/v1/N19-1423

El-Mehalawi, M., & Allen Miller, R. (2003a). A database system of mechanical components based on geometric and topological similarity. part i: Representation. *Computer-Aided Design*, *35*, 83–94. https://doi.org/10.1016/s0010-4485(01)00177-4

El-Mehalawi, M., & Allen Miller, R. (2003b). A database system of mechanical components based on geometric and topological similarity. part ii: Indexing, retrieval, matching, and similarity assessment. *Computer-Aided Design*, *35*, 95–105. https://doi.org/10.1016/s0010-4485(01)00178-6

Geva, M., Schuster, R., Berant, J., & Levy, O. (2021). Transformer feed-forward layers are key-value memories (M.-F. Moens, X. Huang, L. Specia, & S. W.-t. Yih, Eds.). *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 5484–5495. https://doi.org/10.18653/v1/2021.emnlp-main.446

Gümeli, C., Dai, A., & Nießner, M. (2022, June). Roca: Robust cad model retrieval and alignment from a single image. *arXiv.org*. https://doi.org/10.1109/CVPR52688.2022.00399

Hackman, L., Mack, P., & Ménard, H. (2024). Behind every good research there are data. what are they and their importance to forensic science. *Forensic Science International: Synergy*, 100456–100456. https://doi.org/10.1016/j.fsisyn.2024.100456

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, *9*, 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

Ip, C. Y., & Regli, W. C. (2005). Automatic classification of cad models. https://doi.org/10.17918/etd-505

Kanezaki, A., Matsushita, Y., & Nishida, Y. (2018). Rotationnet: Joint object categorization and pose estimation using multiviews from unsupervised viewpoints. *arXiv:1603.06208 [cs]*. Retrieved November 13, 2021, from https://arxiv.org/abs/1603.06208

Kozma, L., & Voderholzer, J. (2024). Theoretical analysis of byte-pair encoding. https://arxiv.org/abs/2411.08671

Kukreja, S., Kumar, T., Bharate, V., Purohit, A., Dasgupta, A., & Guha, D. (2023). Vector databases and vector embeddings-review, 231–236. https://doi.org/10.1109/IWAIIP58158.2023.10462847

Lane, A., & Adelusi, J. B. (2023, July). Role of data classification in enhancing security in big data environments. *ResearchGate*. https://www.researchgate.net/publication/388067206_Role_of_Data_Classification_in_Enhancing_Security_in_Big_Data_Environments

Li, M., Zhang, Y., Fuh, J., & Qiu, Z. (2011). Design reusability assessment for effective cad model retrieval and reuse. *International Journal of Computer Applications in Technology, 2011*, *40*, 3–12. https://doi.org/10.1504/IJCAT.2011.038546

Liu, Y., Tilahun, G., Gao, X., Wen, Q., & Gervers, M. (2025). A comparative study of static and contextual embeddings for analyzing semantic changes in medieval latin charters. Retrieved October 12, 2025, from https://aclanthology.org/2025.loreslm-1.14.pdf

Machalica, D., & Matyjewski, M. (2019). Cad models clustering with machine learning. *Archive of Mechanical Engineering*, *66*(2), 133–152. https://doi.org/10.24425/ame.2019.128441

Mandelli, L., & Berretti, S. (2022, October). Cad 3d model classification by graph neural networks: A new approach based on step format. *arXiv.org*. Retrieved October 13, 2025, from https://arxiv.org/abs/2210.16815

Mason, L. (2019). Data vs information. *www.academia.edu*. https://www.academia.edu/39505535/Data_vs_Information

Maturana, D., & Scherer, S. (2015). Voxnet: A 3d convolutional neural network for real-time object recognition. *2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 922–928. https://doi.org/10.1109/IROS.2015.7353481

Newhouse, W., Souppaya, M., Kent, J., Sandlin, K., & Scarfone, K. (2023). Data classification concepts and considerations for improving data protection. *Data Classification Concepts and Considerations for Improving Data Protection*. https://doi.org/10.6028/nist.ir.8496.ipd

Nguyen, H.-N., & Cuong, L. (2025). Overview of data classification and applications in data security. *International Journal of Environmental Sciences*, *11*, 31–39. https://doi.org/10.64252/ac0b3685

Olson, K. (2021). What are data? *Qualitative Health Research*, *31*(8), Article 1015960. https://doi.org/10.1177/10497323211015960

Omolayo, O., & Babatope, O. (2025). Comparative evaluation of vector embedding frameworks for scalable sematic retrieval in pdf-based rag systems. *International Journal of Research Publication and Reviews*, *6*, 12506–12511. https://doi.org/10.55248/gengpi.6.0625.23100

Pajo, P. (2025). Vector embeddings unveiled: A comprehensive exploration of their creation, types, applications,... *ResearchGate*. https://doi.org/10.13140/RG.2.2.15544.05129

Papadakis, P., Pratikakis, I., Theoharis, T., & Perantonis, S. (2009). Panorama: A 3d shape descriptor based on panoramic views for unsupervised 3d object retrieval. *International Journal of Computer Vision*, *89*, 177–192. https://doi.org/10.1007/s11263-009-0281-6

Patil, R., Boit, S., Gudivada, V., & Nandigam, J. (2023). A survey of text representation and embedding techniques in nlp. *IEEE Access*, *11*, 36120–36146. https://doi.org/10.1109/access.2023.3266377

Qader, W., M. Ameen, M., & Ahmed, B. (2019). An overview of bag of words;importance, implementation, applications, and challenges, 200–204. https://doi.org/10.1109/IEC47844.2019.8950616

Qaiser, S., & Ali, R. (2018). Text mining: Use of tf-idf to examine the relevance of words to documents. *International Journal of Computer Applications*, *181*. https://doi.org/10.5120/ijca2018917395

Qi, C. R., Su, H., Mo, K., & Guibas, L. J. (2016). Pointnet: Deep learning on point sets for 3d classification and segmentation. *arXiv.org*. https://arxiv.org/abs/1612.00593

Rahman, M., Khatib, M., Rani, P., Shaikh, S., & Gunashekar, G. (2019). Effect of computer aided drafting on manual drafting skills. *International Journal of Innovative Technology and Exploring Engineering*, *8*, 3012–3014. https://doi.org/10.35940/ijitee.K2315.0981119

Roj, R., Sommer, M., Woyand, H.-B., Theiß, R., & Dueltgen, P. (2021). Classification of cad-models based on graph structures and machine learning. *Computer-Aided Design and Applications*, *19*, 449–469. https://doi.org/10.14733/cadaps.2022.449-469

Samuels, A. I. J. I. B. W. (2024). One-hot encoding and two-hot encoding: An introduction. https://doi.org/10.13140/RG.2.2.21459.76327

Schoonenboom, J. (2023). The fundamental difference between qualitative and quantitative data in mixed methods research. *Forum Qualitative Sozialforschung / Forum: Qualitative Social Research*, *24*. https://doi.org/http://dx.doi.org/10.17169/fqs-24.1.3986

Sfikas, K., Pratikakis, I., & Theoharis, T. (2018). Ensemble of panorama-based convolutional neural networks for 3d model classification and retrieval. *Computers & Graphics*, *71*, 208–218. https://doi.org/https://doi.org/10.1016/j.cag.2017.12.001

Sfikas, K., Theoharis, T., & Pratikakis, I. (2017). Exploiting the PANORAMA Representation for Convolutional Neural Network Classification and Retrieval. In I. Pratikakis, F. Dupont, & M. Ovsjanikov (Eds.), *Eurographics workshop on 3d object retrieval*. The Eurographics Association. https://doi.org/10.2312/3dor.20171045

Sherstinsky, A. (2020). Fundamentals of recurrent neural network (rnn) and long short-term memory (lstm) network. *Physica D: Nonlinear Phenomena*, *404*, 132306. https://doi.org/10.1016/j.physd.2019.132306

Shivayogi, S. K. (2025). Data classification methodologies and implementation. *Journal of Computer Science and Technology Studies*, *7*, 202–210. https://doi.org/10.32996/jcsts.2025.7.5.26

Suyunu, B., Taylan, E., & Özgür, A. (2024). Linguistic laws meet protein sequences: A comparative analysis of subword tokenization methods. https://arxiv.org/abs/2411.17669

Taipalus, T. (2024). Vector database management systems: Fundamental concepts, use-cases, and current challenges. *Cognitive Systems Research*, *85*, 101216. https://doi.org/https://doi.org/10.1016/j.cogsys.2024.101216

Taylor, P. (2025, June). Data growth worldwide 2010-2028. *Statista*. Retrieved October 12, 2025, from https://www.statista.com/statistics/871513/worldwide-data-created/?srsltid=AfmBOoos3sZoZKw6aS7wf2CiXFwWvkUGV6VUzUBHutfpJE82JTtKg53o

Tran, H. (2024). *Natural language processing for everyone*. Bookdown. https://bookdown.org/tranhungydhcm/mybook/

Trunk, G. V. (1979). A problem of dimensionality: A simple example. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *PAMI-1*, 306–307. https://doi.org/10.1109/TPAMI.1979.4766926

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Proceedings of the 31st International Conference on Neural Information Processing Systems*, 6000–6010.

Voita, E., Talbot, D., Moiseev, F., Sennrich, R., & Titov, I. (2019). Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned. Retrieved October 12, 2025, from https://aclanthology.org/P19-1580.pdf

Wang, B., Wang, A., Chen, F., Wang, Y., & Kuo, C.-C. (2019). Evaluating word embedding models: Methods and experimental results. *APSIPA Transactions on Signal and Information Processing*, *8*. https://doi.org/10.1017/ATSIP.2019.12